Day 92: Training the Mini-GPT on Real Text — Honest Perplexity Numbers

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 14, Day 2 | Complete study guide

Watch: Let's build GPT: from scratch, in code, spelled out (Andrej Karpathy) | Let's reproduce GPT-2 (124M) (Andrej Karpathy)

Script: `day92_training_mini_gpt.py` · every number in this guide is copied by program from `day92_run_output.txt`, the real output of that script.

Day 90's GPT trained properly: a warm-up and cosine schedule, a peak learning rate chosen by a real sweep, a bigger model, and held-out loss measured with full context every 500 steps next to the same measurement on training text. The schedule alone improves Day 90's model (2.443 against 2.492 bits per character). The big model reaches 2.188: just below Day 91's Kneser-Ney counter (2.197) — a tie at best, not a triumph — and its gap to the training text has grown to 0.358 bits. Its perplexity is 4.56 per character, or 4190 per word: the same model, two very different-looking numbers.

0. Where today fits

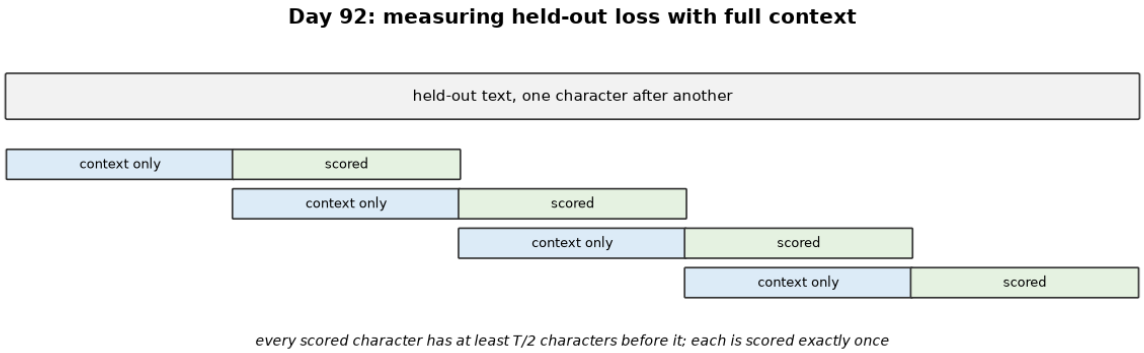
DEFINITION — start here, no prior knowledge assumed

A **learning-rate schedule** changes the step size during training. **Warm-up** starts small and grows, so the first, chaotic updates of a fresh Transformer are gentle; **cosine decay** then shrinks the step smoothly, so the end of training settles instead of bouncing. **Honest numbers** means: measured on held-out text, with the context the model would really have, next to the same measure on training text, and with the unit stated.

REAL-WORLD SCENARIO

Learning to drive: slowly in an empty car park first, faster on the open road, and carefully again when parking.

SYSTEM DIAGRAM



Held-out loss with full context: windows every T/2, only their second halves scored.

Today's part for Day 97

`day92_exercise.py` is the schedule and a full-context evaluator that takes any model as a function, checked so that every position is scored once, a uniform model scores $\log_2 V$ and a model that knows the answer scores 0.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The learning-rate schedule

DEFINITION — start here, no prior knowledge assumed

Linear warm-up over the first 200 steps, then half a cosine from the peak down to 10% of it at step 5,000.

REAL-WORLD SCENARIO

Turning a dimmer up gently, then letting it fade slowly through the evening.

MATHEMATICS

$$lr(s) = peak \cdot (s + 1) / W \text{ for } s < W; \quad \text{afterwards } lr(s) = peak \cdot [f + (1 - f)\,(1 + \cos(\pi\,(s - W) / (S - W))) / 2]$$

$$W = 200 \text{ warm-up steps}, S = 5{,}000 \text{ steps}, f = 0.1$$

IMPLEMENTATION

```
def lr_schedule(step, peak, warmup=WARMUP, total=STEPS, floor=FLOOR):
    if step < warmup:
        return peak * (step + 1) / warmup
    progress = (step - warmup) / max(1, total - warmup)
    return peak * (floor + (1 - floor) * 0.5 * (1 + np.cos(np.pi * min(1.0, progress))))
```

ACTUAL OUTPUT

```
  step: 0: 0.005, 99: 0.500, 199: 1.000, 200: 1.000, 1000: 0.940, 2600: 0.550, 4000: 0.193, 4999:
     0.100
Actual output from this lesson's run.
```

What it means

The rate starts at 0.005 of the peak, reaches 1.000 at the end of the warm-up, is 0.550 of it at step 2,600, the middle of the decay, and 0.100 at the end.

WHY CHOOSE THIS?

The standard schedule for Transformers; two numbers to choose (warm-up, floor).

WHY NOT THIS? (TRADEOFFS)

Fixes the length of training in advance.

2. Choosing the peak learning rate

DEFINITION — start here, no prior knowledge assumed

Three peaks for the big model, 300 steps each (warm-up 100 inside them), all at once, seed 0. A short sweep cannot pick the best rate for 5,000 steps with certainty, but it rules out rates that fail early.

REAL-WORLD SCENARIO

Testing three oven temperatures for ten minutes before committing the whole batch.

ACTUAL OUTPUT

```
  step    lr 0.001    lr 0.003    lr 0.006
    50       5.390       5.141       5.048
   100       4.307       4.031       4.723
   150       3.745       3.709       4.634
   200       3.613       3.602       4.690
   250       3.503       3.479       4.667
   300       3.425       3.374       4.639
chosen for the big model: peak learning rate 0.003 (lowest training loss at step 300)
Actual output from this lesson's run.
```

What it means

0.006 stalls at about 4.639 bits: too large. 0.001 and 0.003 both learn; 0.003 is ahead at step 300 (3.374 against 3.425), so the big model uses it. The margin is small: over 5,000 steps either might have done as well, which this lesson does not test.

WHY CHOOSE THIS?

Cheap, and rules out the failing rate.

WHY NOT THIS? (TRADEOFFS)

Short runs can favour rates that are worse over the full run.

3. Three runs

DEFINITION — start here, no prior knowledge assumed

Day 90's model (2 blocks of 64, context 64) at Day 90's constant rate and with the schedule; and the big model (4 blocks of 128, context 128) with the schedule. Same seed, same 5,000 steps, the same 2,048 characters per step, trained at the same time in three processes.

REAL-WORLD SCENARIO

Three runners on the same track for the same time: one in old shoes, one with a training plan, one bigger and with the plan.

SPECIFICATION

Measured	every 500 steps: held-out bits with full context (Section 0's diagram) and the same on a training-text slice of the same length; mean training-batch loss since the last check.
Models	111,937 and 824,129 weights; Adam (Day 37), clipping (Day 60), Day 90's code.

IMPLEMENTATION

```
def train_run(job):
    name, cfg, peak, schedule, steps, seed = job
    _, _, vocab, tr, va = data()
    tr_eval = tr[:len(va)]
    with threadpool_limits(1, "blas"):
```

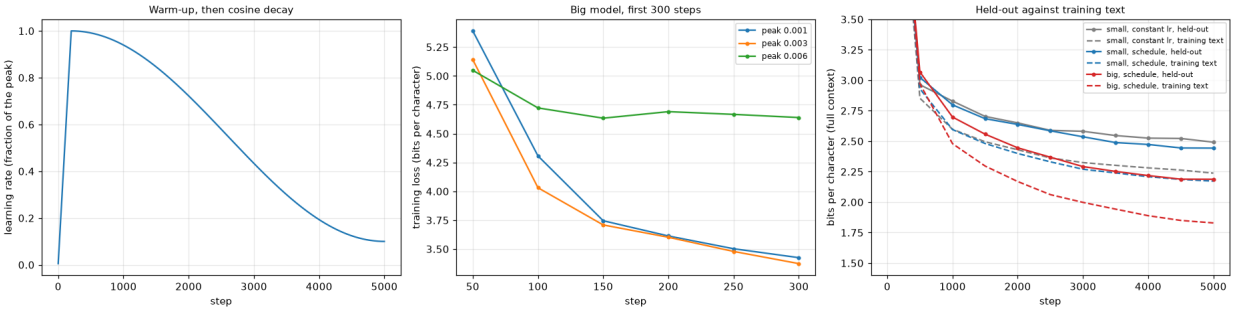
```
        rng = np.random.RandomState(seed)
        P = d90.init_gpt(rng, len(vocab), d=cfg["d"], ff=cfg["ff"], layers=cfg["layers"],
            t=cfg["T"])
        state, recent, busy = {"t": 0, "m": {}, "v": {}}, [], 0.0
        measure = steps > SWEEP_STEPS
        curve = [(0, 0.0, np.nan, eval_bits(P, va, cfg["T"], cfg["h"]), eval_bits(P, tr_eval,
            cfg["T"], cfg["h"]))] if measure else []
        T, B = cfg["T"], cfg["B"]
        for step in range(1, steps + 1):
            t0 = time.perf_counter()
            lr = lr_schedule(step - 1, peak, warmup=min(WARMUP, steps // 3), total=steps) if
                schedule else peak
            o = rng.randint(0, len(tr) - T - 1, B)
            x = np.stack([tr[i:i + T] for i in o]); y = np.stack([tr[i + 1:i + T + 1] for i in o])
            loss, G = d90.gpt_loss_grads(x, y, P, h=cfg["h"])
            d90.adam_step(P, G, state, lr); recent.append(loss)
            busy += time.perf_counter() - t0
            if (measure and step % CHECK_EVERY == 0) or (not measure and step % 50 == 0):
                row = (step, lr, np.mean(recent) / np.log(2))
                if measure:
                    row += (eval_bits(P, va, T, cfg["h"]), eval_bits(P, tr_eval, T, cfg["h"]))
                curve.append(row); recent = []
    return dict(name=name, curve=np.array(curve), P=P, sec=busy / steps, weights=sum(a.size for _,
        a in d90.w12.leaves(P)))
```

ACTUAL OUTPUT

```
small, constant lr: 111937 weights, context 64, peak lr 0.003, 252 ms per step
  step        lr  train batches  train text  held-out
     0   0.00000              -       6.023     6.025
   500   0.00300          3.345       2.852     2.963
  1000   0.00300          2.726       2.598     2.829
  1500   0.00300          2.570       2.495     2.703
  2000   0.00300          2.480       2.430     2.650
  2500   0.00300          2.433       2.362     2.589
  3000   0.00300          2.392       2.324     2.581
  3500   0.00300          2.361       2.302     2.546
  4000   0.00300          2.338       2.281     2.525
  4500   0.00300          2.315       2.263     2.522
  5000   0.00300          2.294       2.238     2.492
```

ACTUAL OUTPUT

```
small, schedule: 111937 weights, context 64, peak lr 0.003, 251 ms per step
  step        lr  train batches  train text  held-out
     0   0.00000              -       6.023     6.025
   500   0.00297          3.661       2.929     3.023
  1000   0.00282          2.773       2.595     2.798
  1500   0.00254          2.574       2.481     2.684
  2000   0.00217          2.466       2.399     2.637
  2500   0.00174          2.402       2.331     2.585
  3000   0.00130          2.349       2.270     2.536
  3500   0.00090          2.305       2.239     2.488
  4000   0.00058          2.269       2.209     2.473
  4500   0.00037          2.239       2.186     2.444
  5000   0.00030          2.222       2.172     2.443
big, schedule: 824129 weights, context 128, peak lr 0.003, 986 ms per step
  step        lr  train batches  train text  held-out
     0   0.00000              -       6.024     6.021
   500   0.00297          3.644       2.971     3.068
  1000   0.00282          2.724       2.479     2.698
  1500   0.00254          2.414       2.296     2.557
  2000   0.00217          2.252       2.169     2.445
  2500   0.00174          2.151       2.061     2.369
  3000   0.00130          2.070       1.998     2.292
  3500   0.00090          2.002       1.942     2.252
  4000   0.00058          1.941       1.889     2.218
  4500   0.00037          1.901       1.850     2.188
  5000   0.00030          1.870       1.829     2.188
Actual output from this lesson's run.
```



Left: the schedule. Middle: the sweep. Right: held-out (solid) and training text (dashed) for the three runs.

What it means

With the same model and steps, the schedule ends at 2.443 against 2.492 at the constant rate; the two are level at step 2,500 (2.585 against 2.589) and the scheduled run leads from step 3,000 on. The big model is better at every check after step 1,000 and ends at 2.188, flat over its last 500 steps. Its training-text loss (1.829) keeps falling faster than its held-out loss: the start of overfitting. Steps cost 986 ms for the big model against 252 ms for the small one on a shared machine; the three runs took 94 minutes together.

WHY CHOOSE THIS?

Every claim backed by a held-out curve with a training-text twin.

WHY NOT THIS? (TRADEOFFS)

One seed per run; the schedule's gain is small enough that seeds could move it.

4. Honest numbers

DEFINITION — start here, no prior knowledge assumed

The final and best held-out loss, the gap to training text, and perplexity per character and per word. Per word: the same model's bits per word, which is bits per character × characters per word.

REAL-WORLD SCENARIO

Quoting a car's fuel use per kilometre or per tank: both true, very different numbers, useless without the unit.

MATHEMATICS

$$\textit{perplexity per character} = 2^{b};\quad \textit{per word} = 2^{b \times \textit{characters per word}}$$

ACTUAL OUTPUT

```
Perplexity = 2^(bits per token): the number of equally likely choices the model is, on average, choosing
    between. Per character it is small; per word it is large. The training text has 5.50 characters per word
    (spaces and newlines included), so bits per word = bits per character x that.
run                  final held-out    best (step)  train text    gap  ppl/char  ppl/word
small, constant lr            2.492   2.492 (5000)       2.238  0.254      5.62     13358
small, schedule               2.443   2.443 (5000)       2.172  0.271      5.44     11103
big, schedule                 2.188   2.188 (5000)       1.829  0.358      4.56      4190
(gap = held-out minus training-text bits: how much worse the model does on text it never saw)
Actual output from this lesson's run.
```

What it means

The best checkpoint is the last one for every run, so nothing is hidden by early stopping. The big model's perplexity is 4.56 per character, as if choosing among about four and a half equally likely characters; per word it is 4190, because a word takes 5.50 characters on average. Neither number can be compared with a perplexity measured in other units, on other text, or without full context. Against Day 91's Kneser-Ney (2.197, scored on slightly different positions), the big model's 2.188 is a tie.

WHY CHOOSE THIS?

Numbers a reader can reproduce and compare.

WHY NOT THIS? (TRADEOFFS)

Honest numbers are less impressive than the usual ones.

Interview preparation — technical questions

Q: Why warm up the learning rate?

A: Early updates of a fresh Transformer are large and erratic; small steps first avoid a bad start.

Q: Why decay it?

A: Smaller steps at the end let the model settle; here the scheduled run ended at 2.443 against 2.492.

Q: How should held-out loss be measured for a fixed-context model?

A: With full context: overlapping windows, only the positions with enough context scored.

Q: Why measure loss on training text too?

A: The gap to held-out shows overfitting: 0.358 bits for the big model.

Q: How do you convert perplexity per character to per word?

A: Raise 2 to bits per character times characters per word.

Interview preparation — theoretical questions

Q: Is a perplexity of 4.56 good?

A: Only in context: per character, on this text, with full context; per word the same model is 4190.

Q: Why did a 0.006 peak fail?

A: It was too large for this model: training stalled at about 4.6 bits. Why exactly was not examined.

Q: Does the big model beat the counting model?

A: Barely: 2.188 against 2.197, on slightly different positions; call it a tie.

Q: What would help next?

A: More data or regularisation, given the growing gap; longer training alone is flattening out.

Key takeaway and what's next

KEY TAKEAWAY

Training choices matter and so does reporting: a schedule improves the same model, a bigger model improves it more, and measured with full context the big mini-GPT reaches 2.188 bits per character, level with a well-smoothed counting model, while the gap to its training text keeps growing. Perplexity without its unit means nothing.

- Small model: constant rate 2.492, schedule 2.443 bits per character.

- Big model (824,129 weights): 2.188; training text 1.829; gap 0.358.
- Perplexity: 4.56 per character, 4190 per word (5.50 characters per word).
- Peak rate: 0.006 fails; 0.003 chosen narrowly over 0.001.

Suggested watch order, if starting from zero

- 1. Let's build GPT: from scratch, in code, spelled out (Andrej Karpathy) — a character-level GPT on Shakespeare, trained step by step.
- 2. Let's reproduce GPT-2 (124M) (Andrej Karpathy) — the training details (schedule, warm-up) of a real GPT.

Day 93 preview

- **Sampling.** The saved big model writes text four ways: greedy, temperature, top-k and top-p, each measured.